Movie Discovery, Review & Cinema Booking Database

Application Design (proposed)

Name	ID No.	Name	ID No.
Sashwath Swaminathan	2024A3PS0361H	Srujana N S	2024A1PS0874H
Anushka Sahoo	2024A1PS0937H	Vinny Polamarasetty	2024A1PS0930H

1. Overview

Phase 1 was all about the database. For the final project we're building a small web app on top of it, basically a mix of Letterboxd and BookMyShow. Users can browse movies, rate and review them, keep a watchlist, and then find a show near them and book seats. This document covers how we're planning to build it. It's a proposal, so some details might change once we start coding.

One idea we're sticking to: **the important rules stay in the database**, not in the app code. So even if the frontend or backend has a bug, Oracle still won't let a seat be sold twice, a rating go above 5, or a payment be marked SUCCESS without a time.

2. Technology Stack

Layer	What we're using	Why
Database	Oracle Database 21c XE, SQL + PL/SQL	required for the course; we already have the schema, constraints, views and trigger working
DB tools	SQL Developer, SQL*Plus	running scripts, testing queries, demoing in the viva
Backend	Node.js + Express	simple to write REST APIs; we know JavaScript already
DB driver	node-oracledb (thin mode)	official Oracle driver, no Instant Client install needed, has connection pooling
Frontend	React (Vite) + Tailwind CSS	component-based, fast dev server; good for the seat map UI
Other	Git + GitHub, Postman, dotenv	version control, testing the API, keeping DB passwords out of the code

3. High-level Architecture

It's a normal 3-tier setup. The React frontend only ever talks to our Express API. The API is the only thing that connects to Oracle, using a connection pool. All queries use bind variables, never string concatenation, so SQL injection isn't possible.

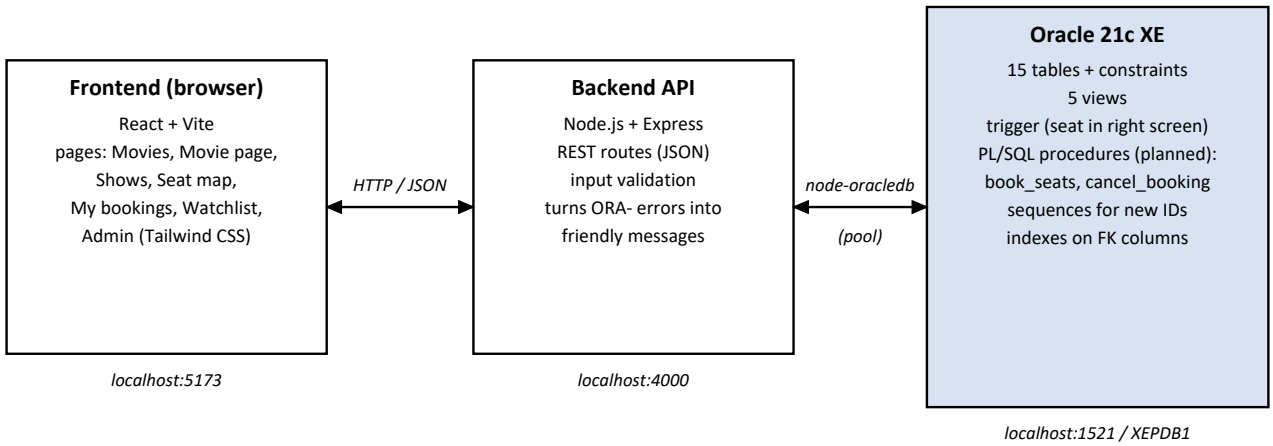


Figure 1: high-level architecture

- **Frontend:** pages and components only, no business rules. It shows whatever the API returns.
- **Backend:** routes → small service functions → SQL. Reads mostly go through our views (e.g. SHOW_SEAT_STATUS, MOVIE_RATING_SUMMARY), so the SQL in the app stays short. Writes for bookings go through PL/SQL procedures.
- **Database:** constraints + trigger are the final safety net. The backend turns Oracle errors into readable messages, e.g. ORA-00001 on UQ_BSEAT_SHOW_SEAT becomes "Sorry, one of those seats was just booked".

4. Main Application Workflows

#	Workflow	What the user does	What happens in the DB
W1	Browse / search movies	filter by genre, language, title; sort by rating	SELECT on MOVIE_DETAILS + MOVIE_RATING_SUMMARY views
W2	Movie page	see details, cast, genres, average rating and reviews	views + REVIEW joined with APP_USER
W3	Rate / review a movie	give 0.5-5 stars, optional text; edit later	MERGE into REVIEW (PK stops duplicates, CHECK stops bad ratings)
W4	Watchlist	add/remove movies; get a hint when one is in theatres	INSERT/DELETE on WATCHLIST; INTERSECT with MOVIE_SHOW (like Q37)
W5	Find shows	pick city and date, see what's playing where	THEATRE_SHOW_SCHEDULE view
W6	Book seats	open seat map, pick seats, confirm	book_seats procedure: BOOKING + BOOKING_SEAT in one transaction
W7	Pay (simulated)	choose UPI / card / etc., get confirmation	INSERT PAYMENT (SUCCESS + paid_at), booking → CONFIRMED
W8	My bookings / cancel	see tickets, cancel an upcoming one	BOOKING_SUMMARY view; cancel_booking frees seats, status CANCELLED
W9	Admin	add movies, theatres, screens and shows	plain INSERTs; UNIQUE (screen_id, show_start) blocks clashes

Booking workflow in detail (W6)

This is the part we were most careful about, because it's where two users can clash:

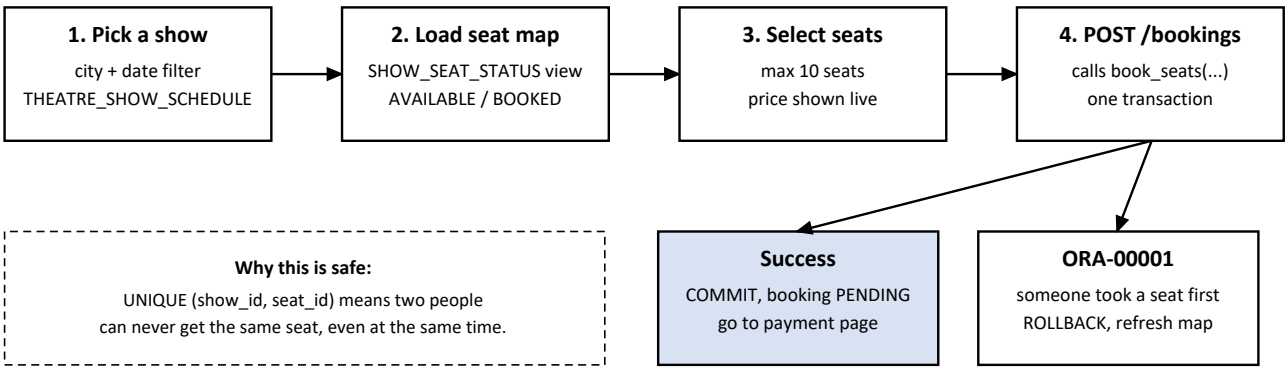


Figure 2: seat booking flow

5. API Endpoints (planned)

Method	Endpoint	Used for
GET	`/api/movies?genre=&lang=&q=`	W1 list / search movies
GET	`/api/movies/:id`	W2 movie page (details, cast, rating, reviews)
PUT	`/api/movies/:id/review`	W3 add or update my review
GET / POST / DELETE	`/api/watchlist`	W4 my watchlist
GET	`/api/shows?city=&date=`	W5 shows by city and date
GET	`/api/shows/:id/seats`	W6 seat map for a show
POST	`/api/bookings`	W6 book selected seats
POST	`/api/bookings/:id/pay`	W7 simulated payment
GET / POST	`/api/bookings/mine, /api/bookings/:id/cancel`	W8 my bookings, cancel
POST	`/api/admin/movies, /api/admin/shows`	W9 admin

For the demo we'll have a simple "pick a user" dropdown instead of real login, since authentication is out of scope for us.

6. What we still need to add to the database

- **PL/SQL procedures:** book_seats (checks that the seats are free, inserts the booking and its seats, computes the premium price) and cancel_booking (deletes the BOOKING_SEAT rows and sets status CANCELLED).

- **A function** like get_seat_price(show_id, seat_id), so the price rule (premium = base + 100) is defined in one place.
- **Indexes** on foreign key columns that the app filters on a lot: BOOKING(user_id), MOVIE_SHOW(movie_id), REVIEW(movie_id).
- **More sample data** so the app has more to show (more shows and dates).

7. Current progress

Part	Status
ER/EER design, relational schema, normalization	done
Tables, constraints, sequences, sample data (02, 03)	done, tested on Oracle 21c XE
37 SQL queries, 5 views, seat/screen trigger (04-06)	done, all results checked (see test report)
PL/SQL procedures, function, indexes	next
Backend API (Express) and frontend (React)	next, after mid-sem